CONTROLLING ACCESS TO FILES

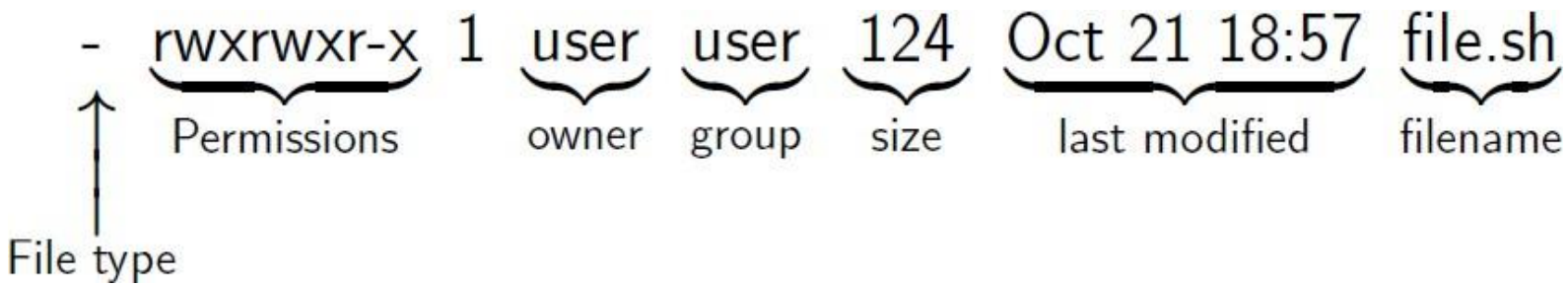
Plan

- **Linux File System Permissions**
- **Managing File System Permissions**

Linux File System Permissions

File properties

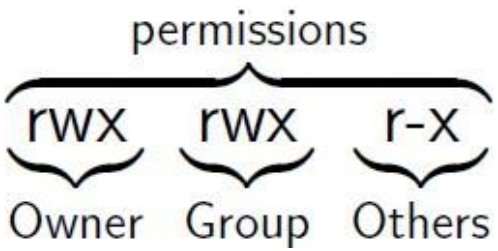
- Recapitulate ls –l
- displays information



Ownership vs Permission

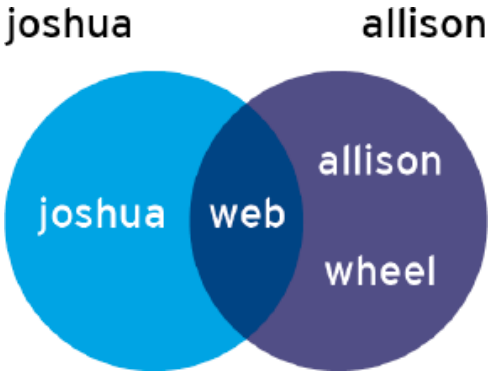
- The file is owned by:
  - a user, normally the one who created the file.
  - a single group, usually the primary groupof the user who created the file

- Ownership: Owner (u), Group (g), Others (o), All (a)
- Permission: Read (r), Write (w), Execute (x)



Effects of Permissions on Files and Directories

PERMISSION	EFFECT ON FILES	EFFECT ON DIRECTORIES
r (read)	Contents of the file can be read.	Contents of the directory (the file names) can be listed.
w (write)	Contents of the file can be changed.	Any file in the directory can be created or deleted.
x (execute)	Files can be executed as commands.	Contents of the directory can be accessed. (You can change into the directory, read information about its files, and access its files if the files' permissions allow it.)



- **read-only directories:** users normally have both read and execute permissions on so that they can list the directory and have full read-only access to its contents.
- If a user only has read access on a directory, the names of the files in it can be listed, but no other information, including permissions or time stamps, are available, nor can they be accessed.
- **execute-only access on a directory**, Users cannot list the names of the files in the directory, but if they already know the name of a file that they have permission to read, then they can access the contents of that file by explicitly specifying the file name
- **Write permission on a directory**: A file may be removed by anyone who has write permission to the directory in which the file resides.

EXAMPLES OF PERMISSION EFFECTS

USER	GROUP MEMBERSHIPS
operator1	operator1, consultant1
database1	database1, consultant1
database2	database2, operator2
contractor1	contractor1, operator2

```
[database1@host dir]$ ls -la
total 24
drwxrwxr-x.  2 database1 consultant1   4096 Apr  4 10:23 .
drwxr-xr-x. 10 root      root          4096 Apr  1 17:34 ..
-rw-rw-r--.  1 operator1 operator1     1024 Apr  4 11:02 lfile1
-rw-r--rw-.  1 operator1 consultant1   3144 Apr  4 11:02 lfile2
-rw-rw-r--.  1 database1 consultant1  10234 Apr  4 10:14 rfile1
-rw-r-----.  1 database1 consultant1   2048 Apr  4 10:18 rfile2
```

EFFECT	WHY IS THIS TRUE?
The user `operator1` can change the contents of **`rfile1`**.	User `operator1` is a member of the `consultant1` group, and that group has both read and write permissions on **`rfile1`**.
The user `database1` can view and modify the contents of **`rfile2`**.	User `database1` owns the file and has both read and write access to **`rfile2`**.
The user `operator1` can view but not modify the contents of **`rfile2`** (without deleting it and recreating it).	User `operator1` is a member of the `consultant1` group, and that group only has read access to **`rfile2`**.
The users `database2` and `contractor1` do not have any access to the contents of **`rfile2`**.	**`other`** permissions apply to users `database2` and `contractor1`, and those permissions do not include read or write permission.
`operator1` is the only user who can change the contents of **`lfile1`** (without deleting it and recreating it).	User and group `operator1` have write permission on the file, other users do not. But the only member of group `operator1` is user `operator1`.
The user `database2` can change the contents of **`lfile2`**.	User `database2` is not the user that owns the file and is not in group `consultant1`, so **`other`** permissions apply. Those grant write permission.
The user `database1` can view the contents of **`lfile2`**, but cannot modify the contents of **`lfile2`** (without deleting it and recreating it).	User `database1` is a member of the group `consultant1`, and that group only has read permissions on **`lfile2`**. Even though **`other`** has write permission, the group permissions take precedence.
The user `database1` can delete **`lfile1`** and **`lfile2`**.	User `database1` has write permissions on the directory containing both files (shown by **`.`**), and therefore can delete any file in that directory. This is true even if `database1` does not have write permission on the file itself.

CHANGING FILE AND DIRECTORY PERMISSIONS

$$\text{chmod}\ \underbrace{\text{<Ownership>}}_{\text{u | g | o | a}}\ \underbrace{\text{<Action>}}_{\text{+ - =}}\ \underbrace{\text{<Permission>}}_{\text{r | w | x}}\ \text{file(s)}$$

- Assign read permission to all: **chmod a=r file1**
- Assign read permission to all: **chmod ugo=r file1**
- Grant execute permission to owner: **chmod u+x file1**
- Deny write access to group and others: **chmod go-w file1**

Examples

- Remove read and write permission for group and other on **file1**:

```
[user@host ~]$ chmod go-rw file1
```

- Add execute permission for everyone on **file2**:

```
[user@host ~]$ chmod a+x file2
```

MANAGING FILE SYSTEM PERMISSIONS

Recursive Permission

```
[root@host opt]# chmod -R g+rwX demodir
```

- the -R option recursively set permissions on the files in an entire directory tree
- The example recursively sets read and write access on demodir and all its children for their group owner.
- It only applies group execute permissions to directories and files that already have execute set for user,group, or other.

Changing Permissions with the Numeric Method

- Uses number (No.) to set permissions (r, w, x) for each Ownership (u, g, o)

$$\text{chmod}\ \overbrace{\underbrace{\langle\text{owner}\rangle}_{\text{No.}}\ \underbrace{\langle\text{group}\rangle}_{\text{No.}}\ \underbrace{\langle\text{other}\rangle}_{\text{No.}}}^{\text{permissions}}\ \text{file(s)}$$

No.	r	w	x
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

- r (all): **chmod 444 file1**
- rw (only owner): **chmod 600 file1**
- rw (owner); r (group & other): **chmod 644 file1**
- rwx (owner); rx (group & other): **chmod 755 file1**
- rwx (all): **chmod 777 file1** – Dangerous

Examples

- Set read and write permissions for user, read permission for group and other, on **`samplefile`**:

```
[user@host ~]$ chmod 644 samplefile
```

- Set read, write, and execute permissions for user, read and execute permissions for group, and no permission for other on **`sampledir`**:

```
[user@host ~]$ chmod 750 sampledir
```

CHANGING FILE AND DIRECTORY USER OR GROUP OWNERSHIP

- Change owner and group
  chown <owner Username>:<group name> <file(s)>
- Change only group membership
  chown :<group name> <file(s)>
- Recursively change: **-R**
- Normal users: Permitted to change only the files they own
- No restriction for super user/root

File ownership can be changed with the **chown** (change owner) command. For example, to grant ownership of the **test_file** file to the student user, use the following command:

```
[root@host ~]# chown student test_file
```

chown can be used with the **-R** option to recursively change the ownership of an entire directory tree. The following command grants ownership of **test_dir** and all files and subdirectories within it to student:

```
[root@host ~]# chown -R student test_dir
```

The **chown** command can also be used to change group ownership of a file by preceding the group name with a colon (:). For example, the following command changes the group **test_dir** to admins:

```
[root@host ~]# chown :admins test_dir
```

The **chown** command can also be used to change both owner and group at the same time by using the *owner:group* syntax. For example, to change the ownership of **test_dir** to visitor and the group to guests, use the following command:

```
[root@host ~]# chown visitor:guests test_dir
```